Sail Declarative Pipelines



```
let mut graph = graph();
register_sql(
    &mut graph,
    "CREATE STREAMING TABLE rows AS SELECT 1",
    "rows.sql",
).await?;
```

Alexy Khrabrov
and Sail Contributors



FIRST PAIR PRESS

Sail Declarative Pipelines

Spark Declarative Pipelines on Sail, the manual

Alexy Khrabrov and Sail Contributors

2026-11-03

Version 1.0.0-bc96ca90
Source commit bc96ca9017ebf7ecddd3cae4404359e6acaf5e1e

First Pair Press
firstpair.press

Contents

1 Introduction 4
1.1 What a pipeline is 4
1.2 What this manual covers 5
1.3 Versions 5
2 Getting started 5
2.1 Build Sail 5
2.2 Install the client 6
2.3 Start a server 6
2.4 Create a project 6
2.5 Run it 6
2.6 Look at the result 7
2.7 Where the server is 7
2.8 Next 7
3 Projects and specs 7
3.1 The spec file 7
3.2 Source files 8
3.3 Where the server is 8
3.4 Layout of the storage root 9
3.5 Names 9
4 Python datasets 9
4.1 Decorators 10
4.2 Streaming tables and append flows 10
4.3 Reading other datasets 11
4.4 Sinks 11
4.5 What a query function may do 12
4.6 Per-flow settings 12
4.7 Mixing with SQL 12
5 SQL pipeline files 12
5.1 Datasets 12
5.2 Reading a stream 13
5.3 Flows 13
5.4 Directives 14
5.5 Order and references 14
5.6 A file registers all or nothing 14
5.7 Comments and separators 15
6 Running 15
6.1 What a run does 15
6.2 Events 15
6.3 Refresh options 16
6.4 Retries 16
6.5 Failures and the rest of the graph 17
6.6 Interrupting a run 17
6.7 Running the same graph twice 17
7 Sources and formats 17
7.1 Tables 17
7.2 Files 17

7.3 Delta tables ... 18
7.4 ORC ... 19
7.5 Parquet, CSV and JSON as targets ... 20
8 Catalogs ... 20
8.1 The memory catalog ... 20
8.2 Configuration ... 20
8.3 Naming datasets ... 21
8.4 Hive metastore ... 21
8.5 Seeing another session's writes ... 22
8.6 Views and comments ... 22
9 Streaming tables ... 22
9.1 Sources ... 22
9.2 What a run reads ... 22
9.3 Checkpoints ... 23
9.4 What the query may do ... 23
9.5 Windowed aggregates ... 23
9.6 Several flows into one table ... 24
9.7 Schema changes ... 24
10 AutoCDC ... 24
10.1 Defining a flow ... 24
10.2 What the target holds ... 25
10.3 Runs ... 26
10.4 Queries between the source and the target ... 27
10.5 Targets ... 27
10.6 What an operator sees ... 28
10.7 Limits in this release ... 29
11 Apache Iggy as a source ... 29
11.1 Reading a topic ... 29
11.2 Positions ... 30
11.3 Topics that change ... 30
11.4 What is refused ... 30
11.5 Running a broker for tests ... 30
12 AutoCDC targets in Unity Catalog and Iceberg ... 31
12.1 Delta tables managed by Unity Catalog ... 31
12.2 Iceberg ... 31
13 Operations ... 32
13.1 Starting a server ... 32
13.2 Cluster mode ... 32
13.3 Sessions ... 33
13.4 Shutdown ... 33
13.5 Logs ... 33
13.6 Storage ... 33
14 Differences from Spark ... 34
14.1 Runs that are stopped ... 34
14.2 Registration ... 34
14.3 Datasets ... 34
14.4 Errors ... 35

14.5 ORC 35
14.6 AutoCDC targets 35
14.7 Sources 35
14.8 What is the same 35
15 The demo 35
15.1 Running it 35
15.2 The projects 36
15.3 The steps 36
15.4 What it found 37
16 Appendix: configuration 37
16.1 Pipeline settings 37
16.2 Table properties 37
16.3 Spec fields 38
16.4 Tool options 38
16.5 Server settings 38
16.6 Source options 39
17 Appendix: error conditions 39
17.1 Registration 39
17.2 Analysis 40
17.3 Execution 40

1 Introduction

Sail Declarative Pipelines is Sail's implementation of Spark Declarative Pipelines, the pipeline framework that ships with Apache Spark 4.2.1. You describe datasets and the queries that fill them, in Python or SQL, and the engine works out the order, runs the queries, publishes the results to the catalog, and remembers how far each streaming source has been read. Sail runs this on its own engine, written in Rust on DataFusion, with no JVM.

The client is the one that ships with Spark: the `pyspark.pipelines` Python API and the `spark-pipelines` command-line tool. Neither is patched. They talk to Sail over Spark Connect, the same way they talk to Spark.

1.1 What a pipeline is

A pipeline is a graph. Its nodes are datasets and its edges are flows.

- A **materialized view** holds the result of a batch query. Every run recomputes it.
- A **streaming table** receives rows from its sources incrementally. Every run reads only what arrived since the last run and appends it.
- A **temporary view** is a named query that other datasets in the pipeline can read. It is never published to the catalog.
- A **persisted view** is published to the catalog as a view, with its SQL text.
- A **sink** is an external destination that a streaming flow writes to.
- A **flow** is a query with a target. A dataset defined with a query has one flow of the same name. A streaming table can have several append flows, and an **AutoCDC** flow applies a change feed to it.

A run takes the whole graph, checks it, and then fills the datasets in dependency order. A dry run stops after the check and writes nothing.

1.2 What this manual covers

This manual describes what works at milestone 1 of Sail Declarative Pipelines, which is the complete batch implementation together with streaming tables, AutoCDC, and Apache Iggy as a streaming source. Everything in it has been run. Where Sail and Spark behave differently on purpose, the chapter on differences says so and why.

The chapters are in the order you would meet things:

1. Getting started: build Sail, install the client, run a first pipeline.
2. Projects and specs: the project layout and the spec file.
3. Python datasets: the decorators and functions of `pyspark.pipelines`.
4. SQL pipeline files: the statements and directives of a `.sql` file.
5. Running: the commands, refresh modes, events and errors.
6. Sources and formats: tables, files, Delta, ORC.
7. Catalogs: the memory catalog, shared memory, a Hive metastore, several catalogs.
8. Streaming tables: how incremental reads and checkpoints work.
9. AutoCDC: applying change feeds.
10. Apache Iggy: reading a topic.
11. Operations: configuration, cluster mode, shutdown.
12. Differences from Spark.
13. The demo: a walkthrough of the example that proves all of the above.

Two appendices list the configuration keys and the error conditions you can see.

1.3 Versions

Component	Version
Apache Spark client (`pyspark[connect]`)	4.2.1
Delta Lake (as a source and a target)	the protocol Sail's Delta support implements
Apache Iggy broker	the revision pinned in the Sail workspace (`153fdf466`)
Python	3.10 or later

Sail itself is built from the `work/spark-declarative-pipelines` branch at the milestone tag.

2 Getting started

This chapter builds Sail, installs the client, and runs a first pipeline on one machine.

2.1 Build Sail

Sail is built from its repository with a Rust toolchain:

```
git clone https://github.com/querygraph/sail
cd sail
git checkout sdp-batch-m1
cargo build --release -p sail-cli
```

The binary is `target/release/sail`.

2.2 Install the client

The client is the unchanged Spark 4.2.1 Python package. No JVM is needed; it talks to Sail over Spark Connect.

```
python3 -m venv .venv
.venv/bin/pip install 'pyspark[connect]==4.2.1' pyyaml
```

This installs the `spark-pipelines` command into `.venv/bin`. `python -m pyspark.pipelines.cli` is the same program.

2.3 Start a server

```
SAIL_CATALOG__SHARED_MEMORY=true target/release/sail spark server --port 15002
```

Port 15002 is where a Spark Connect client looks when it is told nothing else.

`SAIL_CATALOG__SHARED_MEMORY=true` makes every session on this server share one in-memory catalog. Without it each session has a catalog of its own, so the tables a `spark-pipelines run` creates are visible only to that run's session. The setting is off by default; the chapter on catalogs explains the choice, and how to use a Hive metastore instead.

2.4 Create a project

```
.venv/bin/spark-pipelines init --name first
```

This creates:

```
first/
  spark-pipeline.yml
  pipeline-storage/
  transformations/
    example_python_materialized_view.py
    example_sql_materialized_view.sql
```

The spec names the project and points `storage` at `pipeline-storage` as a `file://` URI. The two examples each define a materialized view.

2.5 Run it

From the project directory, check the graph without writing anything:

```
cd first
../.venv/bin/spark-pipelines dry-run
```

Then run it:

```
../.venv/bin/spark-pipelines run
```

The tool prints the pipeline's events as they happen:

```
Flow example_python_materialized_view is QUEUED.
Flow example_sql_materialized_view is QUEUED.
Flow example_python_materialized_view is PLANNING.
Flow example_python_materialized_view is STARTING.
Flow example_python_materialized_view is RUNNING.
Flow example_python_materialized_view has COMPLETED.
```

```
...
Run is COMPLETED.
```

A second `run` recomputes both views; the chapter on running covers the refresh options.

2.6 Look at the result

From any session on the same server:

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder.remote("sc://localhost:15002").getOrCreate()
spark.table("example_python_materialized_view").show()
```

The datasets are ordinary tables in the catalog's warehouse, by default the `spark-warehouse` directory under the server's working directory.

2.7 Where the server is

The tool finds the server through `spark.remote` in the spec's `configuration`, or through the `SPARK_REMOTE` environment variable:

```
SPARK_REMOTE=sc://localhost:15002 ../.venv/bin/spark-pipelines run
```

2.8 Next

- The demo in `examples/declarative-pipelines/` of the Sail repository is a complete project with streaming tables, file and Delta sources, ORC output, AutoCDC and Apache Iggy, with a driver that checks every result. The chapter on the demo walks through it.
- The chapter on projects and specs describes the spec file in full.

3 Projects and specs

A pipeline project is a directory with a spec file and the source files it names. The `spark-pipelines` tool reads the spec, registers every dataset and flow it finds in the sources, and starts a run on the server.

3.1 The spec file

The spec is a YAML file. Named `spark-pipeline.yaml` (or `.yml`) in the current directory, the tool finds it on its own; under any other name, pass it with `--spec`:

```yaml
name: retail
catalog: sail
database: demo
storage: "file:///data/pipelines/retail"
configuration:
  "spark.remote": "sc://localhost:15002"
libraries:
  - glob:
      include: "transformations/**"
```

Field	Required	Meaning
`name`	yes	The pipeline's name, used in messages.
`storage`	yes	Where the pipeline keeps its state: checkpoints and the files of memory sinks. An absolute URI with a scheme (`file://`, `s3://`, ...).
`catalog`	no	The catalog that unqualified dataset names belong to. Defaults to the session's current catalog.
`database` (or `schema`)	no	The database that unqualified dataset names belong to. Defaults to the session's current database.
`configuration`	no	Spark SQL settings applied to the session, as strings. `spark.remote` here tells the tool where the server is.
`libraries`	no	Where the sources are. Each entry is a `glob` with an `include` pattern, relative to the spec file.

A field outside this list is an error (`PIPELINE_SPEC_UNEXPECTED_FIELD`). `name` and `storage` are required.

The storage root must be absolute and carry a scheme. A bare path such as `/data/pipelines` is refused at the start of the run:

```
[PIPELINE_STORAGE_ROOT_INVALID] Pipeline storage root must be an absolute path
with a URI scheme (e.g., file://, s3a://, hdfs://). Got: `/data/pipelines`.
```

3.2 Source files

The `include` patterns select the source files. Two kinds are read:

- **Python files** (`*.py`) are executed with `pyspark.pipelines` imported as `dp` and a `spark` session available. The decorators and functions they call register datasets and flows. The chapter on Python datasets describes them.
- **SQL files** (`*.sql`) are parsed and registered statement by statement. The chapter on SQL pipeline files describes what they may contain.

A project may mix both. A dataset defined in SQL can be read from Python and the other way round, because registration happens before any query runs and names are resolved against the whole graph.

3.3 Where the server is

The tool connects through Spark Connect. It finds the server in one of two ways:

- `spark.remote` in the spec's `configuration`, as above;
- the `SPARK_REMOTE` environment variable, for example `SPARK_REMOTE=sc://localhost:15002`.

The spec wins when both are set.

3.4 Layout of the storage root

Under the storage root the pipeline writes:

```
<storage>/_checkpoints/<catalog>/<database>/<table>/<flow>/<n>/state.json
<storage>/_checkpoints/<catalog>/<database>/<table>/<flow>/<n>/state/<batch>/...
<storage>/_memory_sinks/<flow>/...
```

- `_checkpoints` holds one directory per flow that reads streaming sources. Each full refresh opens a new numbered version `<n>`; ordinary runs continue the latest one. `state.json` records the files, Delta versions and offsets already read, the batch in progress if any, and for a windowed aggregate the watermark. The `state/<batch>` directories hold the rows of windows still open.
- `_memory_sinks` holds the rows written to a `memory` sink, which the session reads back through a temporary view named after the flow.

Nothing else is written there. The datasets themselves live in the catalog's warehouse.

3.5 Names

A dataset's name may have one, two or three parts: `orders`, `demo.orders`, `sail.demo.orders`. One- and two-part names are completed with the spec's `catalog` and `database`. Names are case-insensitive: `Orders` and `orders` are the same dataset, and the catalog stores the lowercase form.

A flow has a one-part name. A dataset defined with a query gets a flow of the same name; an append flow is named after its function, or by `name=`.

Two datasets with the same name in one project is an error (`PIPELINE_DUPLICATE_IDENTIFIERS.OUTPUT`), as are two flows with the same name (`PIPELINE_DUPLICATE_IDENTIFIERS.FLOW`).

4 Python datasets

A Python source file defines datasets with the decorators and functions of `pyspark.pipelines`, imported as `dp`. The file runs once, at registration, with a `spark` session in scope. Each decorated function is a **query function**: it returns a DataFrame, and the DataFrame's plan is what the dataset is filled from. The function body runs at registration; the query runs later, during the run.

```python
from pyspark import pipelines as dp
from pyspark.sql import functions as F


@dp.materialized_view(comment="Customers, written as ORC", format="orc")
def customers_orc():
    return spark.read.table("demo.customers_raw")


@dp.table(comment="Click events from JSON files landing in a directory")
def clicks():
    return spark.readStream.format("json").schema("user INT, page STRING").load("/data/
clicks")


@dp.temporary_view
```

```python
def large_orders():
    return spark.read.table("orders").where("amount >= 100")
```

4.1 Decorators

Decorator	Defines
`@dp.materialized_view`	A materialized view, recomputed by every run. Its query reads batch sources.
`@dp.table`	A streaming table when the query reads a stream (`spark.readStream`), otherwise a materialized view.
`@dp.temporary_view`	A private view other datasets read; never published.
`@dp.append_flow(target=...)`	A flow that appends its query's rows to a streaming table or a sink.

Each decorator can be used bare (`@dp.table`) or with options (`@dp.table(name="st", ...)`). Without `name=`, the function's name is the dataset's name.

`@dp.materialized_view`, `@dp.table` and `dp.create_streaming_table` accept:

Option	Meaning
`name`	The dataset's name, one to three parts.
`comment`	Published as the table's comment.
`spark_conf`	Spark SQL settings for this dataset's flow, as a dict of strings.
`table_properties`	Properties stored on the table.
`partition_cols`	Columns to partition the table by.
`cluster_by`	Clustering columns, recorded on the table. Not together with `partition_cols`.
`schema`	A declared schema, as a DDL string (`"id INT, name STRING"`) or a `StructType`. The query must match it.
`format`	The table's format: `parquet` (the default), `delta`, `orc`, and the other formats Sail writes.

`@dp.temporary_view` accepts `name`, `comment` and `spark_conf`. `@dp.append_flow` accepts `target`, `name` and `spark_conf`.

4.2 Streaming tables and append flows

A streaming table with one query is a `@dp.table` whose function reads a stream:

```python
@dp.table(format="delta")
def delta_events():
    return spark.readStream.format("delta").load("/data/events")
```

A streaming table filled by several flows is created without a query and then targeted:

```
dp.create_streaming_table("all_activity", comment="Orders and clicks, one row per event")


@dp.append_flow(target="all_activity")
def activity_from_orders():
    return spark.readStream.table("orders").select(
        F.col("customer_id").alias("user"), F.lit("order").alias("kind")
    )


@dp.append_flow(target="all_activity")
def activity_from_clicks():
    return spark.readStream.table("clicks").select("user", F.lit("click").alias("kind"))
```

Each flow matches the target's columns by name. A flow is named after its function, or by `name=`; two flows may not share a name.

The `once` option of `append_flow` is not supported, in Sail as in Spark 4.2.1 (`DEFINE_FLOW_ONCE_OPTION_NOT_SUPPORTED`).

4.3 Reading other datasets

Inside a query function, other datasets of the pipeline are read by name, like tables:

- `spark.read.table("orders")` reads a dataset as a batch source.
- `spark.readStream.table("orders")` reads it incrementally, which makes the reading dataset a streaming table.
- `spark.readStream.format("json").schema(...).load(path)` reads files arriving in a directory; `format("delta").load(path)` reads a Delta table incrementally; `format("iggy")` reads an Apache Iggy topic. The chapters on sources and on Iggy have the options.

Names resolve against the pipeline first, then the catalog. A name the pipeline does not define is a catalog table.

4.4 Sinks

A sink is an external destination for streaming flows:

```
dp.create_sink("events_out", format="parquet", options={"path": "/data/out/events"})


@dp.append_flow(target="events_out")
def to_events_out():
    return spark.readStream.table("all_activity")
```

`format` and `options` are the sink's format and its options; a file format needs a `path`. Two sinks need no path: `memory`, whose rows the session can read back through a temporary view named after the flow, and `console`, whose rows are read and discarded.

4.5 What a query function may do

A query function describes a query; it must not run one. Inside it, `spark.sql()` is allowed, and any DataFrame method that builds a plan. A method that needs the server to analyze or execute the plan, such as `.schema`, `.columns`, `.collect()`, `.count()` or `.show()`, is refused:

```
[ATTEMPT_ANALYSIS_IN_PIPELINE_QUERY_FUNCTION] ...
```

Likewise `schema=` must be a DDL string or a `StructType` built in Python, not a schema obtained by analyzing another DataFrame.

4.6 Per-flow settings

`spark_conf` applies to that dataset's flow. A flow that reads a temporary view inherits the view's `spark_conf` too; if the two give one key different values, the run fails with `DUPLICATE_FLOW_SQL_CONF`. Tables contribute nothing to what reads them. The setting most often used this way is the retry count:

```python
@dp.table(spark_conf={"spark.sql.pipelines.maxFlowRetryAttempts": "0"})
def st():
    return spark.readStream.table("src")
```

4.7 Mixing with SQL

A dataset defined in a SQL file is read from Python by its name, and the other way round. Registration happens for every file before the run, so order between files does not matter.

5 SQL pipeline files

A `.sql` file in a project defines datasets and flows with SQL statements. The file is parsed whole, then its statements are registered in order. Registration never runs a query: it records what each dataset is and which query fills it, and the run does the rest.

5.1 Datasets

```sql
CREATE MATERIALIZED VIEW daily_revenue
COMMENT 'Revenue and order count per day'
AS SELECT day, round(sum(amount), 2) AS revenue, count(*) AS orders
FROM orders
GROUP BY day;

CREATE STREAMING TABLE orders
COMMENT 'Orders, read incrementally from demo.orders_raw'
AS SELECT * FROM STREAM demo.orders_raw;

CREATE TEMPORARY VIEW large_orders
AS SELECT * FROM orders WHERE amount >= 100;

CREATE VIEW revenue_report
COMMENT 'Revenue per day, for reporting'
AS SELECT day, revenue FROM daily_revenue;
```

Statement	What it defines
`CREATE MATERIALIZED VIEW name AS query`	A materialized view: a table recomputed by every run.
`CREATE STREAMING TABLE name AS query`	A streaming table whose query reads at least one `STREAM` source.
`CREATE STREAMING TABLE name;`	A streaming table with no query of its own, filled by flows.
`CREATE TEMPORARY VIEW name AS query`	A private view other datasets read; never published.
`CREATE VIEW name AS query`	A persisted view, published to the catalog with its SQL text.

A materialized view and a streaming table accept, between the name and `AS`:

- `COMMENT 'text'`, published as the table's comment;
- `PARTITIONED BY (col, ...)`, identity partitioning on columns of the query;
- `TBLPROPERTIES ('key' = 'value', ...)`;
- a declared column list, `(id INT COMMENT 'the id', value STRING)`, published in that order with those comments. The query's output must match it.

Two things are refused at registration: a `LOCATION`, and a partitioning transform other than a column (`PARTITIONED BY (year(x))` is an "Invalid partitioning transform").

5.2 Reading a stream

A streaming table's query reads its sources with `STREAM`, in either spelling:

```
CREATE STREAMING TABLE st AS SELECT * FROM STREAM(src);
CREATE STREAMING TABLE st AS SELECT * FROM STREAM src;
```

`STREAM` marks the read as incremental: each run reads what was added to `src` since the last run. The source may be a table in the catalog or another dataset of the pipeline, including a temporary view over a streaming read. A materialized view may not read a `STREAM` (`SQL streaming reads cannot back materialized views`), and a persisted view may not either.

5.3 Flows

A flow adds rows to a streaming table from a query of its own. Its statement is `CREATE FLOW` with an `INSERT INTO ... BY NAME`:

```
CREATE STREAMING TABLE all_events;

CREATE FLOW from_orders AS INSERT INTO all_events BY NAME
SELECT customer_id AS user, 'order' AS kind FROM STREAM orders;

CREATE FLOW from_clicks COMMENT 'click events' AS INSERT INTO all_events BY NAME
SELECT user, 'click' AS kind FROM STREAM clicks;
```

- `BY NAME` is required: a flow matches the target's columns by name, not position.
- A flow has a one-part name. `CREATE FLOW a.f` is refused (`MULTIPART_FLOW_NAME_NOT_SUPPORTED`).

- The target must be a streaming table defined in the pipeline (`FLOW_TARGET_NOT_DEFINED` otherwise), and a flow may not share a name with a table (`FLOW_NAME_CONFLICTS_WITH_TABLE`).
- Only `INSERT INTO` is a flow. `CREATE FLOW f AS SELECT 1`, `INSERT OVERWRITE`, `INSERT ... IF NOT EXISTS`, `CREATE TABLE AS` and `EXPLAIN` under a flow are refused.
- A `WITH` clause goes inside the `INSERT`: `INSERT INTO t BY NAME WITH s AS (...) SELECT * FROM s`.

The `AUTO CDC` form of a flow is described in the AutoCDC chapter.

5.4 Directives

Three statements change how the names in the rest of the file are resolved. They apply to the file they are in and are reset at the next file.

```
USE CATALOG sail;
USE SCHEMA demo;
SET CATALOG sail;
```

- `USE SCHEMA name` sets the database that unqualified names belong to.
- `USE CATALOG name` and `SET CATALOG name` set the catalog. `SET CATALOG` takes an expression, as in Spark; a catalog that does not exist is `CATALOG_NOT_FOUND`.
- A bare `USE name` is not a pipeline directive and is refused (`UNSUPPORTED_PIPELINE_SPARK_SQL_COMMAND`).

`SET key = value` sets a Spark SQL configuration for the flows defined after it in the same file. The value is captured per flow; it is not a session setting.

```
SET spark.sql.pipelines.maxFlowRetryAttempts = 0;
CREATE STREAMING TABLE st AS SELECT * FROM STREAM src;
```

5.5 Order and references

Statements may refer to datasets defined later in the same file or in another file. The graph is resolved as a whole, so this is fine:

```
CREATE MATERIALIZED VIEW b AS SELECT * FROM a;
CREATE MATERIALIZED VIEW a AS SELECT * FROM range(1, 4);
```

A persisted view may read tables and other persisted views, but not a temporary view of the pipeline (`INVALID_TEMP_OBJ_REFERENCE`), since the catalog could not resolve the view later. A persisted view's columns must have names: `CREATE VIEW v AS SELECT 1 + 1` is refused (`CREATE_PERMANENT_VIEW_WITHOUT_ALIAS`); write `SELECT 1 + 1 AS two`.

A dataset defined in two files is an error for both (`PIPELINE_DUPLICATE_IDENTIFIERS.OUTPUT`).

5.6 A file registers all or nothing

If any statement of a file fails to register, none of the file's definitions are kept, and the run fails before anything is written:

```
[PIPELINE_SQL_GRAPH_ELEMENT_REGISTRATION_ERROR] Failed to register ... in files.sql
```

The error names the file and the statement. This is stricter than Spark, which keeps the definitions registered before the failing statement; since the client aborts the run in either case, the difference is only what is left registered on the server.

5.7 Comments and separators

Statements are separated by `;`. SQL comments (`--` and `/* */`) are allowed anywhere, and a `;` inside a string literal is not a separator. The text of a persisted view is taken from the file as written.

6 Running

`spark-pipelines` has three commands.

Command	What it does
`spark-pipelines dry-run`	Registers the project and analyzes the whole graph. Writes nothing.
`spark-pipelines run`	Registers, analyzes, then fills every dataset.
`spark-pipelines init --name NAME`	Creates a sample project.

`run` and `dry-run` take `--spec PATH`; without it they look for `spark-pipeline.yaml` or `spark-pipeline.yml` in the current directory.

6.1 What a run does

1. **Registration.** Every source file is read and its datasets and flows are registered. A SQL file that fails registers nothing.
2. **Analysis.** Every flow's query is resolved against the graph and the catalog, in dependency order, and the datasets' schemas are determined. A dry run stops here.
3. **Materialization.** Tables that do not exist are created; tables whose schema changed are altered; tables being fully refreshed are truncated. Persisted views are published.
4. **Execution.** Flows run in dependency order. A materialized view is recomputed; a streaming table's flows read what is new and append it; an AutoCDC flow applies its changes.

Nothing is written before analysis succeeds. If a query cannot be resolved, the run fails with the first problem, and the datasets that depend on it are reported too:

```
Failed to resolve flows in the pipeline.
...
Flow 'broken' has FAILED. Error: [UNRESOLVED_COLUMN.WITH_SUGGESTION] A column ...
```

6.2 Events

The tool prints the server's events as the run proceeds. For each flow:

```
Flow orders is QUEUED.
Flow orders is STARTING.
Flow orders is RUNNING.
Flow orders has COMPLETED.
```

A materialized view's flow also reports `PLANNING` before `STARTING`. A flow left out of a selective refresh reports `Flow 'x' is EXCLUDED.` A flow whose upstream failed reports `Flow 'x' SKIPPED due to upstream failure(s).` The run ends with `Run is COMPLETED.`, or with the error that stopped it. A dry run reports only its end.

6.3 Refresh options

By default every selected dataset is refreshed: a materialized view is recomputed, a streaming table reads what is new.

Option	Effect
`--refresh a,b`	Refresh only the named datasets. The others are excluded.
`--full-refresh a,b`	Reset the named datasets and recompute them from scratch: a streaming table is truncated, opens a new checkpoint, and reads its sources from the beginning.
`--full-refresh-all`	Reset and recompute every dataset.

`--refresh` and `--full-refresh` may be combined for different datasets. Naming a dataset in both is an error, and so is naming any dataset together with `--full-refresh-all`:

```
Cannot specify a subset to refresh when full refresh all is set to true
```

A name may be bare or qualified; `orders` and `demo.orders` are the same dataset.

A selective refresh runs only the selected flows. A selected dataset still reads the current contents of what it depends on; an unselected upstream table is not recomputed first. A persisted view is published whenever its definition is part of the project, selected or not.

6.3.1 Full refresh of a streaming table

A full refresh of a streaming table starts a new checkpoint version. Its flows read every source from the origin, which for a file source means every file in the directory, for a Delta source the table's current snapshot, for an Iggy topic its retained messages. If a source no longer holds its origin, the refresh fails rather than rebuilding a table from a partial history.

A table can opt out of full refreshes with the property `pipelines.reset.allowed` set to `false`. A full refresh of such a table is downgraded to an ordinary refresh, and the table keeps its data.

A full refresh of an AutoCDC target drops its auxiliary state and replays the change feed; the AutoCDC chapter describes it.

6.3.2 Changed streaming sources

A streaming table remembers which streaming sources its flow read. If the flow's sources change between runs (a source added or removed), the run refuses to continue the old checkpoint:

```
Flow 'st' had streaming sources added or removed. It will not be retried.
Please perform a full refresh to rebuild it against the current sources.
```

A full refresh of that table accepts the new source set.

6.4 Retries

A flow that fails is retried, up to `spark.sql.pipelines.maxFlowRetryAttempts` times (default 2, so three attempts). A materialized view is truncated before a retry so a failed attempt leaves no partial result behind. After the last attempt the flow is reported as failed:

```
Flow 'st' has FAILED more than 2 times and will not be restarted.
Error: ...
```

A flow whose source set changed is not retried.

The setting can be given per dataset with `spark_conf` in Python, with `SET` in a SQL file, or for the whole project in the spec's `configuration`.

6.5 Failures and the rest of the graph

When a flow fails for good, the datasets that read from it are skipped, and the rest of the graph runs. The run's result is the first failure. A table whose flow failed may hold a partial result, as in Spark; the next run recomputes a materialized view, and a streaming table replays the batch that did not finish.

6.6 Interrupting a run

Stopping the tool (Ctrl-C) or closing its session interrupts the run on the server: the flows in progress are stopped, and their checkpoints are left at the last completed batch. The next run continues from there. This differs from Spark, which lets the flows that already started finish; the chapter on differences explains why.

6.7 Running the same graph twice

A run is tied to the graph the tool registered. Each `spark-pipelines run` registers the project afresh, so repeated runs are independent. Starting a second run on a graph whose run is still in progress is refused (`EXECUTION_STATE_EXECUTE_HOLDER_ALREADY_EXISTS_GRAPH`).

7 Sources and formats

A pipeline reads from tables, files, Delta tables and Apache Iggy topics, and writes tables in the formats Sail writes. This chapter covers the batch and streaming sources other than Iggy, which has a chapter of its own.

7.1 Tables

Any table the catalog knows is a source. Read it as a batch source with `spark.read.table("t")` or by name in SQL, and as a streaming source with `spark.readStream.table("t")` or `STREAM t`. A streaming read of a table reads its files: the first run reads every file, later runs the files added since.

The datasets of the pipeline are tables too, once the run has filled them. Reading a materialized view with `STREAM` works and reads the files its last refresh wrote.

7.2 Files

A directory of files is read directly:

```
CREATE MATERIALIZED VIEW events AS SELECT * FROM json.`/data/events`;
CREATE MATERIALIZED VIEW counts AS SELECT * FROM parquet.`/data/counts`;
```

```
@dp.table
def clicks():
    return spark.readStream.format("json").schema("user INT, page STRING").load("/data/
clicks")
```

The formats are `parquet`, `orc`, `json` and `csv`. A streaming file read needs a schema. Files whose names begin with `.` or `_` are skipped. An empty location, as in `csv.` `` with nothing between the backticks, is refused: "The location name cannot be empty string".

7.3 Delta tables

A Delta table is read incrementally with the `delta` format, by path or, when the table is in the catalog, by name:

```
@dp.table
def events():
    return spark.readStream.format("delta").load("/data/events")


@dp.table
def events_by_name():
    return spark.readStream.table("delta_events")
```

The first run reads the table's current snapshot. Each later run reads the commits made since, in order. The checkpoint records the Delta version reached. The behavior follows Spark's Delta source, and every rule below was measured on Spark 4.2.1 with Delta 4.4.1:

- A commit that only adds files adds its rows.
- A commit that deletes or updates rows stops the stream with an error, because an append-only consumer cannot apply it: `DELTA_SOURCE_IGNORE_DELETE` for a delete, `DELTA_SOURCE_TABLE_IGNORE_CHANGES` for an update. The options below say what to do instead.
- A commit with no data change (a metadata change, an optimize) adds nothing.
- Deletion vectors are applied: a row deleted by a deletion vector is not read from the snapshot.
- A table that was dropped and recreated at the same path is a different table, and the stream stops rather than mixing the two. A full refresh starts from the new table.

7.3.1 Options

Option	Effect
`ignoreDeletes`	A commit that only deletes rows is skipped. An update still stops the stream.
`ignoreChanges`	A commit that updates rows is read as the files it rewrote, so updated rows appear again (and so do the unchanged rows of those files).
`skipChangeCommits`	A commit that updates or deletes rows is skipped entirely.
`startingVersion`	On the first run, start after this version instead of from the snapshot; `latest` starts from the end, recording nothing until a later run has something to read. Ignored once the stream has a checkpoint.
`startingTimestamp`	Like `startingVersion`, by the first commit at or after the timestamp. A timestamp after the last commit is `DELTA_TIMESTAMP_GREATER_THAN_COMMIT`.
`readChangeFeed`	Read the change data feed instead of the table: each row carries `_change_type`, `_commit_version` and `_commit_timestamp`. The table must have the feed enabled for the versions read.
`excludeRegex`	Files whose paths match the regular expression are not read.
`maxFilesPerTrigger`, `maxBytesPerTrigger`, `withEventTimeOrder`	Accepted and validated as Spark does. A run reads to the end of the log in one batch, so they do not split it.

An invalid option fails the stream with Spark's condition: a version that does not exist is `DELTA_VERSION_NOT_FOUND`; a negative or non-numeric version, or a zero or non-numeric `maxFilesPerTrigger`, is `DELTA_ILLEGAL_OPTION`; both `startingVersion` and `startingTimestamp` is `DELTA_STARTING_VERSION_AND_TIMESTAMP_BOTH_SET`; a timestamp that does not parse is `DELTA_TIMESTAMP_INVALID`. Time travel (`versionAsOf`, `timestampAsOf`) is refused in a streaming read.

7.3.2 Schema

A Delta source's partition columns are columns of the stream. A table with column mapping is read by physical column names, including nested ones. A column added to the source widens the streaming table that reads it. A column renamed or dropped under column mapping stops the stream: the schema it was reading no longer exists.

7.3.3 Delta as a target

A dataset is written as a Delta table with `format="delta"` in Python or `USING delta` in a table definition. A streaming table with a Delta target gets one Delta commit per batch, which is what makes a replayed batch after a crash a second commit of the same rows rather than duplicate files.

7.4 ORC

ORC is read like any file format, and written by giving a dataset `format="orc"`:

```python
@dp.materialized_view(format="orc", table_properties={"compression": "zstd"})
def customers_orc():
    return spark.read.table("demo.customers_raw")
```

The compression codec is taken from `compression`, then `orc.compress`, then the session's default, and is `zstd` when none is given. Sail writes one ORC file per output partition with a single footer.

Timestamps are written as ORC `TIMESTAMP_INSTANT`, which does not depend on the writer's time zone (Spark's choice depends on its JVM's zone). One consequence is deliberate: an instant inside ORC's one-second window before 1970-01-01 fails the write with an error, where Spark writes it and reads back a wrong value.

ORC reads refuse the Hive and forced-positional column-evolution modes (`orc.force.positional.evolution`), and need an object store that reports an ETag or version for each file, which local files and S3 do.

7.5 Parquet, CSV and JSON as targets

Without `format`, a dataset is a Parquet table. `format="csv"` and `format="json"` write those formats. For a file table, partition columns are stored last, in the Spark directory layout.

8 Catalogs

A pipeline's datasets are tables and views in a catalog. Sail's catalogs are configured on the server; the pipeline names the catalog and database its datasets belong to.

8.1 The memory catalog

By default a Sail server has one catalog, named `sail`, of type `memory`, with a database `default`. It keeps table metadata in memory and table data in the warehouse directory, `spark-warehouse` under the server's working directory unless `spark.sql.warehouse.dir` says otherwise.

A memory catalog belongs to a session: each Spark Connect session starts with its own, empty one. The tables a `spark-pipelines run` creates are visible to the run's session, and their files stay in the warehouse, but another session does not see them in its catalog. For a pipeline whose results are read by other sessions, there are two choices:

- **Share the memory catalog.** With `SAIL_CATALOG__SHARED_MEMORY=true`, every session on the server shares one memory catalog, as sessions of one Spark application share Spark's. Another session's inserts and overwrites are visible at once, without a refresh. The catalog still lives only as long as the server.
- **Use a metastore.** A Hive metastore keeps the catalog across servers and restarts.

`catalog.shared_memory` is off by default because an existing Sail deployment may rely on a fresh catalog per session.

8.2 Configuration

Catalogs are configured with environment variables, or the matching keys in Sail's configuration file:

Setting	Meaning	Default
`SAIL_CATALOG__LIST`	The catalogs, as a list of `{name=..., type=..., ...}` entries.	one memory catalog named `sail`
`SAIL_CATALOG__DEFAULT_CATALOG`	The catalog a session starts in.	the first of the list
`SAIL_CATALOG__DEFAULT_DATABASE`	The database a session starts in.	`["default"]`
`SAIL_CATALOG__SHARED_MEMORY`	Share memory catalogs across sessions.	`false`

The list uses a TOML-like syntax. The default is:

```
SAIL_CATALOG__LIST='[{name="sail", type="memory", initial_database=["default"],
initial_database_comment="default database"}]'
```

A server with two memory catalogs:

```
SAIL_CATALOG__LIST='[{name="sail", type="memory", initial_database=["default"]},
{name="other", type="memory", initial_database=["default"]}]'
SAIL_CATALOG__DEFAULT_CATALOG=sail
```

A server whose catalog is a Hive metastore:

```
SAIL_CATALOG__LIST='[{name="sail", type="hms", uris=["127.0.0.1:9083"]}]'
```

The other catalog types, `unity`, `glue` and `iceberg_rest`, are configured with their own fields; AutoCDC's chapter on catalogs covers Unity and Iceberg as targets.

8.3 Naming datasets

A dataset name has up to three parts: `catalog.database.table`. The spec's `catalog` and `database` complete shorter names, and a SQL file's `USE CATALOG`, `SET CATALOG` and `USE SCHEMA` change them for that file. A pipeline may define datasets in several databases and in several catalogs:

```
CREATE MATERIALIZED VIEW first.mv1 AS SELECT 1 AS one;
CREATE MATERIALIZED VIEW other.second.mv2 AS SELECT * FROM first.mv1;
```

Names are resolved against the pipeline before the catalog: a dataset the pipeline defines shadows a catalog table of the same name.

A catalog that does not exist is `CATALOG_NOT_FOUND`. A database the pipeline writes to must exist; the run does not create it.

8.4 Hive metastore

With a `hms` catalog, Sail creates the pipeline's tables in the metastore and keeps their data under the metastore's warehouse. Tables Sail creates are external tables marked `external.table.purge`, so `DROP TABLE` and `DROP DATABASE ... CASCADE` delete their data, as Spark's do. A `file:/path` location is read as an absolute path.

Every behavior in this manual was run against a Hive 4.0.1 metastore as well as the memory catalog; the only differences are the metastore's own: it cannot replace a table in place, so a full refresh of an AutoCDC target there drops and recreates it.

8.5 Seeing another session's writes

Sail serves table reads from the files in the table's directory. When sessions share a catalog, a write from one session is visible to a read from another at once. Two caches can delay that:

- `REFRESH TABLE t` is never needed for visibility in the default configuration. It clears every file-listing and statistics cache entry, since those caches are not keyed by table.
- If the optional file-listing cache is enabled (`execution.file_listing_cache.type`), a listing is reused for `execution.file_listing_cache.ttl` seconds, 1800 by default, and a write from another session is seen after that or after `REFRESH TABLE`. `TRUNCATE TABLE` clears its own directory's entry.

8.6 Views and comments

A materialized view and a streaming table are published as tables, with the `COMMENT` and `TBLPROPERTIES` of their definitions and the declared columns in order. A persisted view is published with its SQL text; `SHOW CREATE TABLE` shows it. A temporary view is never published.

9 Streaming tables

A streaming table is filled incrementally. Each run reads what its sources received since the last run, in one bounded batch, and appends the result. There is no long-running stream: a run is Spark's `availableNow` trigger, executed to completion, and the next run picks up where it left off.

9.1 Sources

A flow of a streaming table reads one or more streaming sources:

Source	How it is read	Progress kept
A table in the catalog, `spark.readStream.table("t")` or `STREAM t`	The table's files. A run reads the files that were not read before.	The set of files read.
A directory of files, `spark.readStream.format("json").schema(...).load(path)`	Files in the directory, in `json`, `csv`, `parquet` or `orc`.	The set of files read.
A Delta table, `spark.readStream.format("delta").load(path)`	The table's snapshot on the first run, then each new commit.	The Delta version reached.
An Apache Iggy topic, `spark.readStream.format("iggy")`	The topic's messages.	The offset per partition.
Another streaming table or a temporary view over a stream	As the dataset's own source.	Per flow.

A file source needs a schema; a table source has one. A directory with no files on the first run is refused, since nothing says what its rows look like. Files whose names start with `.` or `_` are ignored, as Spark ignores them.

Reading a materialized view with `STREAM` is allowed and reads the files the view's last refresh wrote.

9.2 What a run reads

A streaming table's flow is given a batch of what is new: the files not yet read, the Delta commits after the last version, the Iggy messages after the last offset. The batch is written to the table, and only then is the progress recorded. If a run is interrupted after the batch was recorded as pending but before it was written, the next run replays exactly that batch.

This gives **at-least-once** delivery: a batch is never lost, and a batch whose write finished but whose record did not can be written twice. For a plain file table that means duplicate rows after a crash at that moment; for a Delta target the second write is a second commit of the same rows.

Overwriting a source table does not re-read its old rows: the files are new, so they are read, and the files that disappeared are forgotten.

9.3 Checkpoints

Progress lives under the pipeline's storage root, one directory per flow:

`<storage>/_checkpoints/<catalog>/<database>/<table>/<flow>/<version>/state.json`

`version` is `0` for the first run and goes up by one at each full refresh. An ordinary run continues the latest version. `state.json` records the batches completed, what was read, the batch in progress if any, and the watermark of a windowed aggregate. Deleting the directory makes the next run start over, reading every source from its origin into a table that still holds the old rows; use a full refresh instead.

9.4 What the query may do

The query of a streaming table transforms its stream row by row: projections, filters, column expressions, and joins with batch tables. Spark's rules for append-mode streaming apply, and some shapes are refused up front rather than run wrongly:

Shape	Allowed
Join of a stream with a batch table (inner, left with the stream on the left, right with the stream on the right)	yes; the batch side is read at each run
Join of two streams	no
Full outer join involving a stream	no
`GROUP BY` over a stream without a watermark	no: `streaming aggregations without watermark`
`GROUP BY window(...)` with a watermark on the read	yes, see below
`DISTINCT` over a stream	no
Window functions, `ORDER BY`, `LIMIT` over a stream	no
`UNION` of a stream with a batch query	no, as in Spark

The refusals are reported when the run starts, before anything is written.

9.5 Windowed aggregates

A streaming table may aggregate over a tumbling window with a watermark:

```
@dp.table
def counts():
    return (
        spark.readStream.table("events")
        .withWatermark("ts", "1 second")
        .groupBy(F.window("ts", "2 seconds"))
```

```
        .count()
    )
```

Rows are held until the watermark passes the end of their window, then aggregated once and appended. Between runs the open windows' rows are kept under the checkpoint. The rules are Spark's:

- The watermark is the latest event time seen minus the delay, in whole milliseconds. It starts at 0 and never moves back.
- A row is late, and dropped, only when its window has already ended by the watermark the batch started with. A row behind the watermark whose window is still open is counted.
- A window is written once, when it closes. A later run never updates it.

At milestone 1 the window must be tumbling (size and slide equal), with no start offset, and the watermark must be placed directly on the streaming read, with one watermark per query. Sliding windows, several watermarks, deduplication and stream-stream joins are supported for AutoCDC flows (see that chapter), not yet for append flows.

9.6 Several flows into one table

A streaming table may be the target of several append flows. Each keeps its own checkpoint, and each run reads each flow's sources independently:

```
CREATE STREAMING TABLE all_activity;
CREATE FLOW from_orders AS INSERT INTO all_activity BY NAME SELECT ... FROM STREAM
orders;
CREATE FLOW from_clicks AS INSERT INTO all_activity BY NAME SELECT ... FROM STREAM
clicks;
```

9.7 Schema changes

A streaming table's schema is the union of what its flows produce, merged with what the table already has. A new column in a flow's query is added to the table; later rows have it, earlier rows read it as null. A column that a flow stops producing stays in the table and is written as null. A column whose type changes incompatibly fails the run (`CANNOT_MERGE_INCOMPATIBLE_DATA_TYPE`). A declared schema is exact: the query must produce it.

10 AutoCDC

AutoCDC applies a feed of changes to a table. Each change row carries the key of the row it changes, a sequencing value that orders changes to the same key, and, if the feed has deletes, a marker that says the row is gone. Sail keeps the latest row per key, which Spark calls SCD type 1.

This chapter covers what you write, what you see in the target, how runs and refreshes behave, which queries may stand between the source and the target, and what an operator needs to know about the state Sail keeps.

10.1 Defining a flow

An AutoCDC flow is defined in Python with `create_auto_cdc_flow`. Spark 4.2.1 has no SQL form for it, and neither has Sail: `CREATE FLOW ... AS INSERT INTO` is an append flow, not AutoCDC.

```
from pyspark import pipelines as dp
from pyspark.sql import functions as sf
```

```python
@dp.temporary_view
def changes():
    return (
        spark.readStream.format("json")
        .schema("id INT, name STRING, seq BIGINT, is_delete BOOLEAN")
        .load("/data/landing")
    )

dp.create_streaming_table("customers", format="delta")

dp.create_auto_cdc_flow(
    target="customers",
    source="changes",
    keys=["id"],
    sequence_by="seq",
    apply_as_deletes=sf.expr("is_delete"),
    except_column_list=["is_delete"],
)
```

The arguments:

Argument	Meaning
`target`	A streaming table declared with `create_streaming_table`. Give it a format: `delta` or `iceberg`.
`source`	A streaming view, or a table read as a stream, that yields the change rows.
`keys`	The columns that identify a row. Several keys are allowed.
`sequence_by`	The column that orders changes to one key. A later value wins.
`apply_as_deletes`	An expression; a change row for which it is true deletes the key. Optional.
`column_list`	Keep only these columns in the target. Optional.
`except_column_list`	Keep every column but these; usually the delete marker. Optional.
`stored_as_scd_type`	Only `1`. SCD type `2` is refused with `AUTOCDC_SCD2_NOT_SUPPORTED`.
`name`	The flow's name; the target's name when omitted.

The key and sequence columns must exist in the source, and a column cannot be both excluded and a key. These are checked when the pipeline is defined, with the same conditions Spark raises.

10.2 What the target holds

After a run the target holds one row per key: the row with the highest sequence value seen, with the columns the flow keeps. One more column, `__spark_autocdc_metadata`, is a struct of `deleteSequence` and `upsertSequence`; Spark adds it too.

```
+---+-----+---+--------------------------+
|id |name |seq|__spark_autocdc_metadata  |
+---+-----+---+--------------------------+
|1  |bob  |3  |{NULL, 3}                 |
+---+-----+---+--------------------------+
```

Ordering follows the sequence value, not arrival:

- A change whose sequence is lower than the one already applied to its key changes nothing, in this run or a later one.
- Two changes to one key with the same sequence value are a tie. Spark leaves the result undefined; Sail keeps the last one it read.
- A delete removes the row. A later upsert with a higher sequence brings the key back; an upsert with a lower sequence than the delete is ignored, because the delete is remembered.

Deletes are remembered in an auxiliary table beside the target, named `__spark_autocdc_aux_state_<target>`. You can read it; do not write it. Dropping it forgets the deletes it held, so an older upsert for a deleted key would then land; Spark behaves the same way after the table is dropped.

10.3 Runs

A run reads what the source holds when the run starts and applies it in batches of at most 10,000 rows per source partition. The run ends when it has applied everything up to that point; new data that arrived during the run waits for the next run. This is the behaviour of Spark's `AvailableNow` trigger.

Progress lives under the pipeline's storage root:

`<storage>/_checkpoints/<target>/<flow>/<n>/`

`n` counts full refreshes. A later run continues from the recorded progress, so a file read in one run is never read again, and a source file deleted after it was read is not missed unless you ask for a full refresh.

10.3.1 Full refresh

`spark-pipelines run --full-refresh customers` (or `--full-refresh-all`) rebuilds the target from the source's beginning:

1. The target's rows are removed.
2. The auxiliary table is dropped and created empty, so remembered deletes are forgotten, as Spark does.
3. Any operator state (see below) starts empty.
4. Progress moves to a new checkpoint directory, `<n+1>`.

A source that no longer holds its beginning refuses a full refresh before anything is removed, for instance a Delta source whose first versions were vacuumed, or an Iggy topic whose retention moved on.

A target with the Spark table property `pipelines.reset.allowed` set to `false` is not rebuilt; the run becomes an ordinary one, as in Spark.

10.3.2 Selective refresh

`--refresh customers` runs only the flows into that target; other tables keep their contents.

10.4 Queries between the source and the target

The flow's source can be more than a read: a view that filters, projects, joins, aggregates or deduplicates streams. Spark runs these as a stream with a state store; Sail keeps the equivalent state between batches and runs, under the checkpoint directory. These forms are supported:

Query	Behaviour
Filters, projections, added or renamed columns	Stateless.
`withWatermark(...).groupBy(window(...), ...)`	A window is emitted once, when the watermark passes its end. Tumbling and sliding windows, with or without a start time. One stream or several.
`dropDuplicates()` and `dropDuplicates([cols])`	A key seen in any earlier batch or run is dropped.
Stream joined with a table	Stateless; the table is read at each batch.
Two streams joined (inner)	Both sides are kept. A pair is emitted once, when its later row arrives.

Spark's watermark rule is followed exactly: the watermark is the latest event time seen, in whole milliseconds, less the delay, and it never moves back. A row is late only when its window has already ended by the watermark; a row behind the watermark whose window is still open still counts. With several watermarked streams the query's watermark is the smallest, and a stream that has produced nothing yet counts as zero. After a restart each stream's watermark starts again, as in Spark, so a window closes only when every stream has moved in the new run.

These are refused, with Spark's error where Spark has one:

- An aggregate over a stream without a watermark: `STREAMING_OUTPUT_MODE.UNSUPPORTED_OPERATION`.
- `dropDuplicatesWithinWatermark`.
- A left join whose right side is a stream, a right join whose left side is a stream, and a full join over any stream. Window functions, sorts and limits over a stream.
- A union of a stream with a table.

A changed query over existing state is refused: if the columns or types that a deduplication or window keeps are not the ones in the state, the run stops with a message that names the flow and asks for a full refresh.

10.5 Targets

Target	How Sail writes it
Delta, by path or in the memory, Hive or second catalog	Fenced: every write goes through the control store described below. The table refuses `ALTER TABLE` and any write that does not come from the pipeline.
Delta managed by Unity Catalog	Plain: Unity orders the commits. See the catalogs chapter.
Iceberg	Plain: Iceberg orders the commits. New columns arrive by `ALTER TABLE ... ADD COLUMNS`, which Sail issues; a full refresh drops and recreates the table.

A target created before the first run is adopted on that run. A target you drop and create again while enrolled is refused by the next ordinary run, with a message that says to run a full refresh; the full refresh adopts the new table.

10.5.1 Schema evolution

A new source column is added to the target before the next batch. Nested fields are added inside their struct, and for arrays of structs inside the element. A column whose type changed is refused with Spark's `DATATYPE_MISMATCH`; a full refresh, which recreates the target, takes the new type.

10.6 What an operator sees

10.6.1 The control store

For a fenced Delta target, Sail records every batch before its first effect and every write's receipt in a small database:

```
<state root>/_control/authority.db
```

The state root is the warehouse directory. For a remote warehouse, such as `s3://bucket/warehouse`, it is a local directory instead, named by `SAIL_AUTOCDC_STATE_DIR` or, when that is unset, `~/.sail/autocdc/<sha256 of the warehouse URL>`. One machine owns a remote warehouse's flows. The warehouse carries a marker file, `_autocdc_state_owner`, naming that state directory by an id the directory keeps. A machine that finds the marker but not the directory, or another directory, stops with a message that says what to restore, so a flow never starts over against tables that have history. Removing the marker and running a full refresh is the way to adopt the warehouse on a new machine.

10.6.2 Leases and recovery

A run holds a lease on each target for as long as it runs, renewed every five seconds, with a fifteen-second term. If the process dies, the next run waits for the lease to lapse, at most about seventeen seconds, then resumes the unfinished batch from the recorded state and repeats only the writes that have no receipt. Nothing is written twice. A run that fails for an ordinary reason releases the lease at once, so the next run, often the full refresh the error asked for, does not wait.

Two sessions cannot run the same flow at once: the second waits on the lease, then recovers whatever the first left unfinished, and the first can no longer write.

Process death is tested, including kills at every stage of a batch. Loss of power, and restoring the state directory from a backup, are not qualified.

10.6.3 Enrolled tables

An enrolled Delta table carries the property `sail.autocdc.authority`. Any Sail session, on any server, refuses to write such a table unless the write comes from the pipeline that enrolled it:

```
table file:///.../customers/ is enrolled with AutoCDC authority ... and refuses a write
without its fence
```

`ALTER TABLE` is refused the same way, whether the table is named through a catalog or by its path (`ALTER TABLE delta.`/path/to/customers`...`). To change the table yourself, drop it and run a full refresh, which recreates it; a table created again at the same location is not the enrolled one, and nothing refuses writes to it until the refresh adopts it.

10.7 Limits in this release

- SCD type 2 is refused.
- A join of two streams keeps every row of both sides for good; Spark evicts join state only with watermarks on both sides, which Sail does not do yet.
- `dropDuplicates` on some columns keeps an arbitrary row among the duplicates within one batch, as Spark does within one microbatch.
- Iceberg and Unity targets are not fenced. Two pipelines writing one such target at once each keep their own progress; Spark has the same exposure.
- On an Iceberg target, Spark's own connector refuses three nested-type changes during a full refresh or evolution that Sail completes.

11 Apache Iggy as a source

An Apache Iggy topic can feed an AutoCDC flow. Sail reads it as a bounded stream: each run takes the messages that exist when the run starts, and the next run continues from where the last one ended.

11.1 Reading a topic

```
@dp.temporary_view
def changes():
    return (
        spark.readStream.format("iggy")
        .option("url", "iggy://iggy:secret@127.0.0.1:8090")
        .option("stream", "cdc")
        .option("topic", "events")
        .option("timeout_ms", "5000")
        .schema("id BIGINT, name STRING, op STRING, seq BIGINT")
        .load()
    )

dp.create_streaming_table("target", format="delta")
dp.create_auto_cdc_flow(
    target="target",
    source="changes",
    keys=["id"],
    sequence_by="seq",
    apply_as_deletes=sf.expr("op = 'D'"),
    except_column_list=["op"],
)
```

A schema is required: Sail decodes each message into it.

Option	Meaning
`url`	`iggy://user:password@host:port`. The user and password are used to log in; only the host and port name the source.
`stream`, `topic`	The Iggy stream and topic. Both must exist before the first run.
`payload_format`	`json` (the default), `raw`, or `envelope`.
`source_id`	With `envelope`, the column that holds the envelope's source id.
`batch_size`	Messages asked for in one poll. Default 1000.
`timeout_ms`	The longest one call to the broker may take. Default 30000. Without it a dead broker would be retried without limit.

Option names are case-insensitive, as in Spark. An option that belongs to the Iggy reader's other modes, such as a consumer group, is refused rather than ignored: a bounded read keeps its own positions and never commits an offset to the broker.

11.2 Positions

The first run starts at the first readable offset of each partition and ends at the offset the broker reports when the run starts. Each later run starts where the last ended. Messages that arrive while a run is in progress wait for the next run.

A full refresh starts again from the first readable offset. If the broker's retention has moved past the offset a run needs, the run stops before writing anything; the message names the gap.

11.3 Topics that change

Iggy gives a topic an incarnation; deleting and recreating a topic gives it a new one. A run that finds the topic's incarnation changed stops and leaves the target as it was, because the recorded offsets no longer mean anything. A full refresh adopts the new topic and rebuilds the target from its beginning.

A topic that does not exist is refused before the first write, with a message that says the topic is absent.

11.4 What is refused

- Reading an Iggy topic as a batch, with `spark.read`, is refused; the source serves pipeline AutoCDC flows.
- An append flow (`CREATE FLOW ... AS INSERT INTO`) can read Iggy as well, through the same source; see the streaming chapter.

11.5 Running a broker for tests

Sail's tests run a broker they start themselves. They need the server binary of the Iggy version Sail is built against, named by an environment variable:

```
export SAIL_IGGY_SERVER_BINARY=/path/to/iggy-server
```

Build it from the Iggy repository at the commit Sail pins, with `cargo build --release -p server`, and point the variable at `target/release/iggy-server`. Without the variable the Iggy tests are skipped with a reason that says so. The tests start the broker on free ports with QUIC and WebSocket disabled, create the stream and topic over HTTP, and send messages to partition 0.

12 AutoCDC targets in Unity Catalog and Iceberg

AutoCDC writes Delta tables by path and in Sail's own catalogs under a fence, as the AutoCDC chapter describes. Two other targets take a plain path, in which the catalog or the table format orders the commits and Sail keeps its progress in the checkpoint, as Spark does.

12.1 Delta tables managed by Unity Catalog

With a Unity Catalog configured as a Sail catalog, a streaming table named in that catalog is a managed Delta table whose commits Unity ratifies.

```
dp.create_streaming_table("unity.default.customers", format="delta")

dp.create_auto_cdc_flow(
    target="unity.default.customers",
    source="changes",
    keys=["id"],
    sequence_by="seq",
    apply_as_deletes=sf.expr("is_delete"),
    except_column_list=["is_delete"],
)
```

What differs from a fenced Delta target:

- Unity orders the commits, so there is no lease and no control store for the target. The checkpoint is the progress.
- A full refresh drops the table and creates it again, because Unity does not replace a table in place; the new table has a new id in Unity.
- `DROP TABLE ... PURGE` is not accepted by Unity; Sail drops without it.
- Two pipelines running one flow into one table at once each keep their own progress. Nothing stops the second; the data converges because rows win by sequence, but remembered deletes may interleave.

The auxiliary table lives in the same schema as the target, as `__spark_autocdc_aux_state_<table>`.

The catalogs chapter describes how a Unity Catalog is attached to a Sail server. Sail's own test suite runs against Unity Catalog built from source; `PYSAIL_INTEGRATION_SUITE=unity` selects the suite and `PYSAIL_UNITY_CATALOG_URL` points it at a running server.

12.2 Iceberg

A streaming table with `format="iceberg"` is an Iceberg table.

```
dp.create_streaming_table("events_current", format="iceberg")
dp.create_auto_cdc_flow(target="events_current", source="changes", keys=["id"],
sequence_by="seq")
```

- Iceberg orders the commits; the checkpoint is the progress, and a batch replayed after a crash is safe because rows win by sequence.

- A new source column arrives by `ALTER TABLE ... ADD COLUMNS`, which Sail issues before the batch. Nested fields are added inside their struct or array element.
- A full refresh drops and recreates the table, so a changed key or type takes effect then.
- The table carries no fence. An ordinary write from another session is not refused.

Sail completes three nested-type changes that Spark's Iceberg connector refuses: a full refresh that changes keys and types, a selective full refresh, and a new field inside an array of structs. On Spark those runs fail with `Cannot update '<column>', not a primitive type`.

13 Operations

This chapter covers running the server that pipelines run on: its settings, execution modes, sessions, and shutdown.

13.1 Starting a server

```
sail spark server --port 15002
```

Settings are environment variables of the form `SAIL_<SECTION>__<KEY>`, or the matching keys in Sail's configuration file. The ones a pipeline deployment meets:

Setting	Meaning
`SAIL_CATALOG__LIST`, `SAIL_CATALOG__DEFAULT_CATALOG`, `SAIL_CATALOG__DEFAULT_DATABASE`, `SAIL_CATALOG__SHARED_MEMORY`	The catalogs; see the catalogs chapter.
`SAIL_MODE`	`local` (the default): queries run in the server process. `local-cluster`: the server starts worker processes on the same machine and runs queries across them. `kubernetes-cluster`: workers are Kubernetes pods.
`SAIL_EXECUTION__DEFAULT_PARALLELISM`	How many partitions a query runs with.
`SAIL_RUNTIME__STACK_SIZE`	The stack size of the runtime's threads, in bytes. Deep plans, such as a wide AutoCDC merge, need `16777216`.
`SAIL_IGGY_SERVER_BINARY`	Only for Sail's own test suite: where to find an Iggy broker to start.

The warehouse, where the memory catalog keeps table data, is `spark-warehouse` under the server's working directory, or the `spark.sql.warehouse.dir` of the session.

13.2 Cluster mode

In `local-cluster` and `kubernetes-cluster` modes a pipeline runs the same way: the server is the driver, analysis and the catalog stay on it, and the queries of each flow run on the workers. Every behavior in this manual was run in `local-cluster` mode as well as `local`.

Two things differ for the operator:

- A streaming table's checkpoint is written by the driver; the workers never touch it.

- When a session that used checkpoints is released, the driver asks the workers to stop. If it cannot confirm that every worker writing on the session's behalf has stopped, it does not delete the session's checkpoint files, and the release reports that failure. The files are left for the next run, and the server exits 1 at shutdown. Never deleting what a live writer may still be using is the rule; the message says which prefix was kept.

A worker that is told to stop finishes within 5 seconds, and a driver waits at most 30 seconds for its workers. These bounds are backstops; a stop that hits them is logged.

13.3 Sessions

Each `spark-pipelines` invocation, and each `SparkSession.builder.remote(...)` in Python, is a Spark Connect session. Sessions are independent: a graph registered in one is not visible from another, and a run belongs to the session that started it.

Closing a session (`spark.stop()`, or the tool exiting) stops its run, as the chapter on running describes, and cleans up what the session owned: its checkpoint writers, its memory sinks' files, its streams. A later request on a released session is refused:

```
[INVALID_HANDLE.SESSION_CLOSED] The handle <id> is invalid. Session was closed. SQLSTATE:
HY000
```

Releasing a session twice, or releasing one the server never saw, succeeds.

13.4 Shutdown

`SIGINT` (Ctrl-C) stops the server. It closes every session, waits for their cleanups, and stops the drivers and workers. The exit status is 0 when every cleanup succeeded. A cleanup that failed, for example because a checkpoint prefix could not be confirmed deletable, makes the exit status 1, with the failure in the log; nothing is deleted on a guess.

A job failure that was already reported to its query is not a shutdown failure: it is logged, not repeated. A commit whose outcome is unknown is reported, since only the operator can decide what to do with it.

A server started from a shell that blocks `SIGINT` inherits the block and cannot be stopped with it; send `SIGTERM`, or start it from a shell that does not block the signal.

13.5 Logs

The server logs to standard error. A pipeline run logs its flows' events at `INFO`, and the events are also sent to the tool, which prints them. Session creation and removal are logged by the session manager; a canceled job in cluster mode is logged by the driver with its job id.

13.6 Storage

The pipeline's storage root (`storage` in the spec) and the warehouse may be local directories or object stores (`s3://` and the others Sail's object store support covers). Checkpoint writes are single atomic puts, which is what makes a crash mid-run recoverable on both. Remote warehouses are tested against an S3-compatible store; AutoCDC's own state directory is described in its chapter.

14 Differences from Spark

Sail Declarative Pipelines follows Spark 4.2.1. Every behavior in this manual was measured on Spark first, and where Sail differs it is on purpose. This chapter lists the differences a user can see.

14.1 Runs that are stopped

Spark	Sail
Interrupting a run cancels the operation; the flows that already started run to their end.	The flows in progress are stopped.
Releasing a session during a run lets its flows finish.	The flows are stopped.

Sail stops the flows because otherwise nothing could stop a run, and a server cannot shut down with flows still writing. A stopped flow leaves its checkpoint at its last completed batch; the next run continues from there.

14.2 Registration

Spark	Sail
A SQL file whose statement fails keeps the definitions registered before it.	A failed SQL file registers nothing.
A `CREATE MATERIALIZED VIEW` or `CREATE STREAMING TABLE` run directly as a query writes the `INSERT` before rejecting the statement.	The statement is rejected before anything is written.
Table names with dots or hyphens, `` `a.b` `` or `` `my-table` ``, are refused by the session catalog.	They are accepted, as everywhere in Sail.

In each case Spark's client aborts the run too; the difference is what is left behind on the server.

14.3 Datasets

Spark	Sail
A column added to a streaming table is placed where a Scala `Map` puts it, which varies past four columns.	Added columns follow the target table's order.
A `writeStream` to the `memory` sink outside a pipeline is supported.	It is not. Inside a pipeline, a `memory` sink works.
Flow failure events are emitted in the order of a Scala `Map`, which is unspecified.	They are emitted sorted by name, the downstream group before the direct one.

14.4 Errors

Spark	Sail
`DATA_SOURCE_NOT_FOUND` for a missing format reaches the client wrapped in `TableMaterializationException` inside an `UnknownException`.	The same message, as the error itself.
The merge error for incompatible column types ends with its `SQLSTATE`.	The same text; the `SQLSTATE` suffix is rendered as Spark renders it.

14.5 ORC

Timestamps are written as ORC `TIMESTAMP_INSTANT`. Spark's stored type depends on the time zone of the JVM that writes, which a Sail worker has no equivalent of. One visible consequence: an instant within the one-second window before 1970-01-01 that ORC cannot represent fails the write on Sail, where Spark writes it and reads back a wrong value.

14.6 AutoCDC targets

Spark 4.2.1 refuses a Delta table as an AutoCDC target (`AUTOCDC_TARGET_DOES_NOT_SUPPORT_MERGE`), because its Delta library exposes no row-level operations. Sail accepts Delta targets; this is a deliberate extension, and the usual way to run AutoCDC on Sail.

Spark with Iceberg 1.13 refuses three AutoCDC cases that change nested types. Sail completes them.

14.7 Sources

Apache Iggy is a source on Sail only; Spark has no Iggy source.

14.8 What is the same

Everything else this manual describes: the registration protocol, the validation order and conditions, flow configuration fusion, refresh selection, retries, the events, the checkpoint rules of streaming tables, the Delta source's options and conditions, the SCD1 semantics of AutoCDC, and the watermark rules of windowed aggregates. The frozen test suite ported from Spark, 300 selected public tests and 67 upstream tests, runs on every change, and the pipelines test files are rerun on Spark itself: at milestone 1, only the differences above fail there.

15 The demo

The Sail repository carries a demo at `examples/declarative-pipelines/`. It runs real pipeline projects through the unchanged `spark-pipelines` tool against a Sail server, then checks every result through a Spark Connect session of its own and prints PASS or FAIL for each check. Its exit status is 0 only when every check passed. It is the proof that the features in this manual work together, on one machine, after a checkout and a build.

15.1 Running it

From the repository root, after `cargo build --release -p sail-cli` and a Python environment with `pyspark[connect]==4.2.1` and `pyyaml`:

```
PYTHON=.venv/bin/python examples/declarative-pipelines/run.sh
```

With an Apache Iggy broker, the Iggy checks run too:

```
IGGY_SERVER_BINARY=/path/to/iggy-server PYTHON=.venv/bin/python examples/declarative-pipelines/run.sh
```

Without a broker those checks are reported as SKIP, never as PASS. The run takes a few minutes. It works in a new temporary directory, deleted when every check passes; `--keep` or `--workdir DIR` keeps it, with the server and tool logs.

`run.sh` starts the server with `SAIL_CATALOG__SHARED_MEMORY=true`, so the tool's session and the checking session see one catalog. It also unblocks `SIGINT` before starting, so the server can be stopped at the end; a server started from a shell that blocks the signal cannot be.

15.2 The projects

Project	File	What it shows
`retail`	`orders.sql`	A streaming table over a table, a materialized view, a temporary view, a persisted view, comments.
	`sources.py`	An ORC materialized view, JSON-file and Delta streaming sources, two append flows into one streaming table.
	`cdc.py`	AutoCDC from a JSON change feed into a Delta table, with deletes and sequencing.
`broken`	`broken.sql`	A graph with an unresolved column.
`iggy`	`iggy.py`	A streaming table and an AutoCDC target reading one Iggy topic.

The driver, `demo.py`, writes each project's spec with the server's address and fills in the `{{...}}` placeholders in the Python files with paths in its work directory. The project files are otherwise ordinary SDP sources.

15.3 The steps

1. **Setup.** Two source tables (`demo.orders_raw`, `demo.customers_raw`), a directory of JSON click files, a directory of JSON change files, and a Delta table.
2. **Dry run.** The graph is analyzed and no dataset is created.
3. **First run.** Every dataset is created and checked: the streaming table read its source; the materialized view aggregates it; the temporary view feeds a count but is not published; the persisted view is queryable; comments are published; the ORC view is stored as `.orc` files; the JSON and Delta sources were read; the two append flows filled one table; AutoCDC applied the change feed, with the later sequence winning.

4. **Second run.** A new order, a new click file, a new Delta commit and new changes arrive. Each streaming table appended only what was new; the materialized view was recomputed; AutoCDC applied a delete and an insert and ignored an out-of-order update.
5. **Third run.** With no new data, every streaming table and the AutoCDC target are unchanged.
6. **Selective refresh.** `run --refresh daily_revenue` recomputes that view from the current `orders` and leaves `orders` alone; a normal run then picks up the waiting order.
7. **Full refresh.** After the source table is overwritten, `run --full-refresh-all` rebuilds every streaming table from its sources and replays the change feed to the same AutoCDC state.
8. **Broken graph.** The run fails with `UNRESOLVED_COLUMN`, and the valid dataset in the same graph is not created either.
9. **Iggy.** Three runs over one topic: the first reads it; the second reads only the new messages, applying a delete and an insert and ignoring a stale update; the third reads nothing.
10. **Shutdown.** The server stops cleanly on `SIGINT`.

At the milestone the demo reports 48 checks passed, 0 failed, 0 skipped with a broker.

15.4 What it found

The demo's first run failed every AutoCDC step: Sail's default warehouse is the relative path `spark-warehouse`, and the AutoCDC commit path could not turn a relative location into a URL. No test had seen it, because every test fixture sets an absolute warehouse. The fix resolves the location the way Delta's own table source does. It is a good example of what the demo is for: it runs the configuration a user would, not the one the tests do.

16 Appendix: configuration

16.1 Pipeline settings

These are Spark SQL settings. They can be given for the whole project in the spec's `configuration`, for one dataset with `spark_conf` in Python, or for the flows after a `SET` in a SQL file.

Key	Meaning	Default
`spark.remote`	Where the server is, for the `spark-pipelines` tool.	`sc://localhost:15002`
`spark.sql.pipelines.maxFlowRetryAttempts`	How many times a failed flow is retried.	`2`

16.2 Table properties

Property	Meaning
`pipelines.reset.allowed`	`false` exempts a table from full refreshes: a full refresh of it is downgraded to an ordinary refresh.
`pipelines.autocdc.keyColumnNames`, `pipelines.autocdc.scdType`	Recorded on an AutoCDC target by its flow; see the AutoCDC chapter.

16.3 Spec fields

Field	Meaning
`name`	The pipeline's name. Required.
`storage`	The storage root, an absolute URI with a scheme. Required.
`catalog`, `database` (`schema`)	Where unqualified names belong.
`configuration`	Spark SQL settings, as strings.
`libraries[].glob.include`	Source file patterns, relative to the spec.

16.4 Tool options

Command and option	Meaning
`run --spec PATH`	The spec file; otherwise `spark-pipeline.yaml` or `.yml` in the current directory.
`run --refresh a,b`	Refresh only these datasets.
`run --full-refresh a,b`	Reset and recompute these datasets.
`run --full-refresh-all`	Reset and recompute everything.
`dry-run [--spec PATH]`	Analyze only.
`init --name NAME`	Create a sample project.

16.5 Server settings

Environment variables of the form `SAIL_<SECTION>__<KEY>`.

Variable	Meaning	Default
`SAIL_CATALOG__LIST`	The catalogs.	one memory catalog named `sail`
`SAIL_CATALOG__DEFAULT_CATALOG`	The catalog sessions start in.	the first in the list
`SAIL_CATALOG__DEFAULT_DATABASE`	The database sessions start in.	`["default"]`
`SAIL_CATALOG__SHARED_MEMORY`	Share memory catalogs across sessions.	`false`
`SAIL_MODE`	`local`, `local-cluster` or `kubernetes-cluster`.	`local`
`SAIL_EXECUTION__DEFAULT_PARALLELISM`	Partitions per query.	the number of cores
`SAIL_RUNTIME__STACK_SIZE`	Runtime thread stack size in bytes.	the runtime's default
`SAIL_EXECUTION__FILE_LISTING_CACHE__TYPE`	An optional file-listing cache.	`none`
`SAIL_EXECUTION__FILE_LISTING_CACHE__TTL`	Its entries' lifetime in seconds.	`1800`

16.6 Source options

Delta source options (`ignoreDeletes`, `ignoreChanges`, `skipChangeCommits`, `startingVersion`, `startingTimestamp`, `readChangeFeed`, `excludeRegex`, `maxFilesPerTrigger`, `maxBytesPerTrigger`, `withEventTimeOrder`) are in the chapter on sources. Iggy source options (`url`, `stream`, `topic`, `timeout_ms`) are in the Iggy chapter. ORC write options (`compression`, `orc.compress`) are in the chapter on sources.

17 Appendix: error conditions

The conditions a pipeline can raise, with Spark's names. The client exposes a condition through `getCondition()` on the exception, and the message carries it in brackets.

17.1 Registration

Condition	When
`PIPELINE_SQL_GRAPH_ELEMENT_REGISTRATION_ERROR`	A statement of a SQL file could not be registered; the file registers nothing. The message names the file and statement.
`PIPELINE_DUPLICATE_IDENTIFIERS.OUTPUT`	Two datasets have the same name.
`PIPELINE_DUPLICATE_IDENTIFIERS.FLOW`	Two flows have the same name.
`PIPELINE_DATASET_WITHOUT_FLOW`	A streaming table has no query and no flow.
`MULTIPART_FLOW_NAME_NOT_SUPPORTED`	`CREATE FLOW a.f`.
`FLOW_TARGET_NOT_DEFINED`	A flow's target is not a dataset of the pipeline.
`FLOW_NAME_CONFLICTS_WITH_TABLE`	A flow and a table share a name.
`INVALID_FLOW_QUERY_TYPE`	A sink's flow has a batch query.
`UNSUPPORTED_PIPELINE_SPARK_SQL_COMMAND`	A statement a SQL file may not contain, such as a bare `USE`.
`INVALID_STATEMENT_OR_CLAUSE`	`INSERT ... IF NOT EXISTS` under a flow.
`PARSE_SYNTAX_ERROR`	A statement that does not parse.
`INVALID_TEMP_OBJ_QUALIFIER`	A temporary view qualified with something other than `session` or `system.session`.
`TEMP_VIEW_NAME_TOO_MANY_NAME_PARTS`	A temporary view with a three-part name.
`REQUIRES_SINGLE_PART_NAMESPACE`	A namespace with more than one part where one is required.
`DEFINE_FLOW_ONCE_OPTION_NOT_SUPPORTED`	`append_flow(once=...)`.
`UNSUPPORTED_FEATURE.CREATE_PIPELINE_DATASET_QUERY_EXECUTION`	`CREATE MATERIALIZED VIEW` or `CREATE STREAMING TABLE` run as an ordinary query.
`UNSUPPORTED_FEATURE.CREATE_FLOW_QUERY_EXECUTION`	`CREATE FLOW` run as an ordinary query.
`ATTEMPT_ANALYSIS_IN_PIPELINE_QUERY_FUNCTION`	A query function asked the server to analyze or run a plan.
`PIPELINE_SPEC_UNEXPECTED_FIELD`	An unknown field in the spec.
`PIPELINE_STORAGE_ROOT_INVALID`	A storage root without a scheme.

17.2 Analysis

Condition	When
`UNRESOLVED_COLUMN`, `UNRESOLVED_TABLE_OR_VIEW`, `TABLE_OR_VIEW_NOT_FOUND`	A query refers to something that does not exist.
`AMBIGUOUS_REFERENCE`	A query reads a column that a view repeats.
`INVALID_TEMP_OBJ_REFERENCE`	A persisted view reads a temporary view.
`CREATE_PERMANENT_VIEW_WITHOUT_ALIAS`	A persisted view has an unnamed column.
`MATERIALIZED_VIEW_WITH_MULTIPLE_QUERIES`	A materialized view has more than one flow.
`DUPLICATE_FLOW_SQL_CONF`	A flow and a view it reads set one key to different values.
`CANNOT_MERGE_INCOMPATIBLE_DATA_TYPE`	A streaming table's flows, or a flow and the table, disagree on a column's type.
`USER_SPECIFIED_AND_INFERRED_SCHEMA_NOT_COMPATIBLE`	A declared schema does not match the query.
`COLUMN_NOT_DEFINED_IN_TABLE`	A partition or cluster column is not in the query.
`ALL_PARTITION_COLUMNS_NOT_ALLOWED`	Every column is a partition column.
`SPECIFY_CLUSTER_BY_WITH_PARTITIONED_BY_IS_NOT_ALLOWED`	Both `cluster_by` and `partition_cols`.
`CANNOT_UPDATE_PARTITION_COLUMNS`	A refresh would change an existing table's partition columns.
`UNABLE_TO_INFER_PIPELINE_TABLE_SCHEMA`	A dataset's schema could not be determined.
`STREAMING_OUTPUT_MODE.UNSUPPORTED_OPERATION`	A streaming query shape append mode does not allow, such as an aggregate without a watermark.
`CATALOG_NOT_FOUND`	`SET CATALOG` or a name names a catalog that does not exist.
`DATA_SOURCE_NOT_FOUND`	A `format` no source provides.
`PATH_NOT_FOUND`	A file path that does not exist.

17.3 Execution

Condition	When
`TABLE_MATERIALIZATION_FAILED`	A table could not be created or altered.
`RUN_EMPTY_PIPELINE`	A run with no dataset to refresh.
`SPARK_CONNECT_ILLEGAL_STATE.EXECUTION_STATE_EXECUTE_HOLDER_ALREADY_EXISTS_GRAPH`	A second run on a graph whose run is in progress.
`STREAM_FAILED`	A streaming source failed; the message says why (for a Delta source, one of the `DELTA_*` conditions of the sources chapter).
`INVALID_HANDLE.SESSION_CLOSED`	A request on a released session.
`DIVIDE_BY_ZERO`, `REMAINDER_BY_ZERO`, `CAST_INVALID_INPUT`, `NUMERIC_VALUE_OUT_OF_RANGE`	ANSI arithmetic and casts, as in Spark.

AutoCDC's conditions (`AUTOCDC_*`) and Iggy's are listed in their chapters.